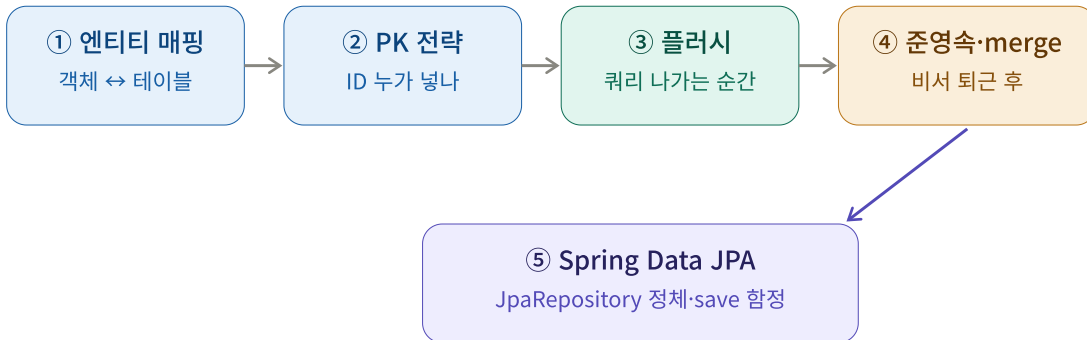


[Lv2] JPA 2주차 학습 노트

EntityManager에서 JpaRepository까지

1주차가 "영속성 컨텍스트라는 엔진의 원리"였다면, 2주차는 "그래서 코드를 실제로 어떻게 짜고 무엇을 조심하나"라는 핸들과 페달이다.



1주차 엔진 위에서 "실제로 코드를 어떻게 짜고 뭘 조심하나"를 배운다

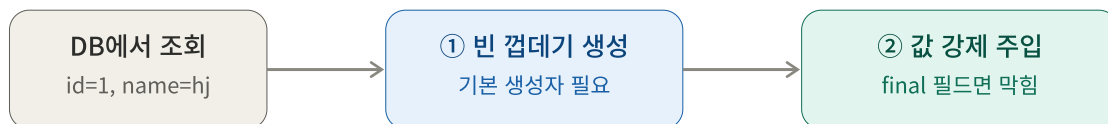
▲ 2주차 전체 흐름

① 엔티티 매핑 — "너는 이제 평범한 객체가 아니야"

엔티티(Entity): DB 테이블과 짝지어진 자바 클래스. `@Entity` 를 붙여야 JPA가 관리하고, `@Id` 로 PK를 지정한다.

리플렉션(Reflection): 실행 중(runtime)에 클래스 정보를 들여다보고 조작하는 자바 기술. "돌아가는 중에" 빈 객체를 만들고 필드에 값을 밀어넣을 수 있다.

JPA는 DB 데이터를 객체로 만들 때: ① 기본 생성자로 빈 객체 생성 → ② 리플렉션으로 필드에 값 강제 주입. 이 순서가 3가지 규칙을 강제한다.



이 순서가 3가지 규칙을 강제한다

- 기본 생성자 필수 — ①에서 객체를 못 만들면 에러
- final 필드 금지 — ②에서 값 주입이 막힘
- final 클래스 금지 — 프록시(가짜 객체)를 상속으로 못 만들

▲ 리플렉션 기반 객체 생성 순서가 세 규칙의 이유

Lombok 해결책: `@NoArgsConstructor(access = AccessLevel.PROTECTED)` . `public` 은 너무 열려서 `new User()` 로 객체를 실수로 만들 수 있고, `private` 은 너무 닫혀서 JPA 프록시가 상속을 못 함. `protected` 가 딱 적당한 균형.

```
@Entity
@Table(name = "TB_USERS") // 테이블 이름
public class User {
    @Id
    private Long id;
    @Column(name = "user_name", nullable = false, length = 100) // 컬럼명 + NOT NULL + 길이
    private String username;
}
```

주요 속성: **name** (컬럼명), **nullable** (기본 true), **unique**, **length** (기본 255). DB엔 없고 객체에서만 쓸 임시 필드는 **@Transient** 로 제외.

★ 실무 경고: EnumType은 무조건 STRING!

자바 **Enum** 을 DB에 저장할 때 **@Enumerated** 를 쓰는데 기본값이 **ORDINAL** (위험). ORDINAL은 순서 숫자로 저장한다.

처음: enum { USER, ADMIN }

USER → 0
DB에 0 저장

ADMIN → 1
DB에 1 저장

나중: enum { GUEST, USER, ADMIN } — 맨 앞에 GUEST 추가

GUEST → 0
이제 0의 의미

USER → 1
1로 밀림

ADMIN → 2
2로 밀림

대참사: DB에 0으로 저장됐던 기존 USER 회원들이

전부 GUEST로 강등된다. 숫자 0은 그대로인데 그 0의 "뜻"이 바뀌었기 때문. → 무조건 STRING

▲ ORDINAL의 데이터 오염 — 순서가 밀리면 기존 데이터의 의미가 바뀐다

```
@Enumerated(EnumType.STRING) // "USER" 문자열 자체 저장 → 순서 바뀌어도 안전
private RoleType role;
```

참고: **ddl-auto** 는 운영에서 **create / update** 쓰면 데이터 삭제 위험 → **none / validate** 만. 방언(Dialect)은 DB 종류별 SQL 문법을 자동 번역.

② PK 생성 전략 — "ID, 내가 넣을까 니가 넣을까"

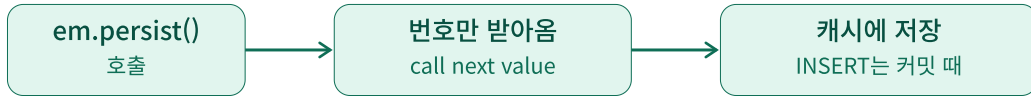
직접 할당(**@Id** 만) vs 자동 생성(**@GeneratedValue** , 4전략). 핵심은 IDENTITY vs SEQUENCE.

IDENTITY (MySQL)



→ 1만 건이면 네트워크 1만 번 왕복 (Batch Insert 불가)

SEQUENCE (Oracle · PostgreSQL · H2)



→ INSERT 없이 ID 확보 → 쓰기 지연 O, Batch Insert O

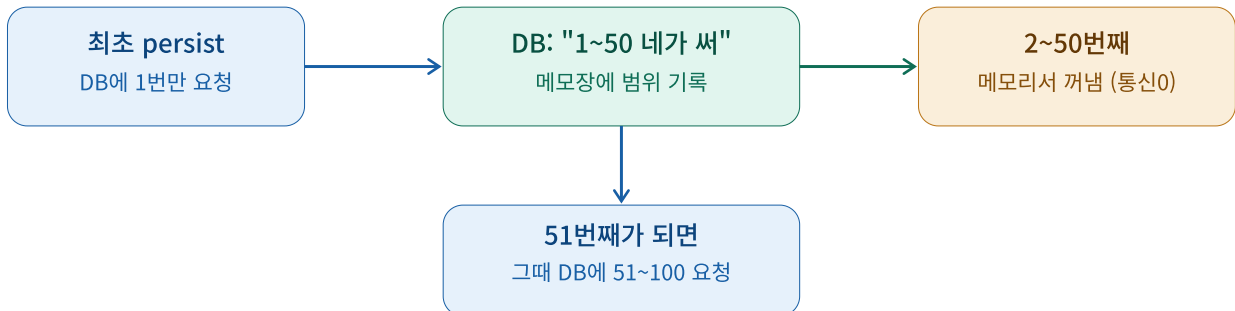
차이의 핵심: SEQUENCE는 "번호만 물어보는 기능"이 따로 있다
IDENTITY엔 그게 없어서 INSERT를 실제로 날려야만 ID가 나온다

▲ IDENTITY는 쓰기 지연 불가, SEQUENCE는 가능 — 결정적 차이는 "번호만 묻기"의 유무

IDENTITY가 쓰기 지연이 안 되는 이유: 1차 캐시가 `Map<ID, 엔티티>` 라 ID가 있어야 넣는데, AUTO_INCREMENT는 INSERT를 날려야만 ID가 나오므로 persist 순간 즉시 INSERT.

★ allocationSize — 네트워크 1/50 압축

```
@SequenceGenerator(name="SEQ_GEN", sequenceName="MY_SEQ",  
    initialValue=1, allocationSize=50) // 기본값 50
```



결과: 50개 저장에 DB 호출 단 1번 → 네트워크 1/50로 압축

▲ 시퀀스를 도매로 확보 — 범위 안에서는 메모리에서 꺼내 통신 0회

⚠ 중복 에러 함정 JPA `allocationSize` 와 DB 시퀀스 `INCREMENT BY` 값이 다르면 PK 중복 에러. 둘을 맞춰야 함 (둘 다 50, 또는 둘 다 1).

TABLE은 별도 테이블로 시퀀스 흉내 → 락 때문에 성능 최악(거의 안 씀). AUTO는 DB에 맞춰 자동 선택하나 실무는 명시적 지정 선호.

왜 단점 있는 IDENTITY가 대세일까?

- MySQL 생태계 — 무료·가벼움·사실상 표준(전통적으로 시퀀스 없었음)

- 대부분 단건 입력 — 회원가입 1명, 주문 1건. 1건은 성능 차이 없음 ("성급한 최적화는 만악의 근원")
- 진짜 대량 입력은 JdbcTemplate — 엑셀 10만 건은 그 부분만 JDBC로

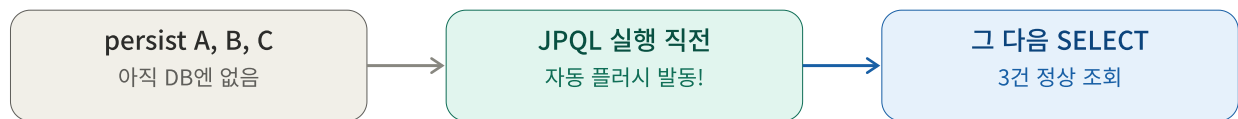
③ 플러시(Flush) — "쿼리가 DB로 날아가는 순간"

플러시: 영속성 컨텍스트(메모리)의 변경 내용을 DB로 반영(동기화). 쓰기 지연 저장소의 SQL을 DB로 쏜.

⚠ 플러시 ≠ 커밋 Flush는 SQL을 DB로 보냄(아직 롤백 가능), Commit은 보낸 걸 확정.

플러시 3가지 경우: ① `em.flush()` 수동 ② 트랜잭션 커밋 ③ JPQL 실행 직전(핵심!).

JPQL: 테이블이 아니라 객체를 대상으로 하는 쿼리(`select m from Member m`). JPA가 실제 SQL로 번역해 DB에 직접 날린다.



만약 자동 플러시가 없다면?

A·B·C가 DB에 없는 상태로 SELECT → 0건 조회 → "방금 저장했는데 왜 안 나와?" 버그

플러시 3가지 경우

① `em.flush()` 수동 ② 트랜잭션 커밋 ③ JPQL 실행 직전 (이게 핵심!)

▲ JPQL 직전 자동 플러시 — 메모리에만 있는 데이터를 먼저 DB로 밀어 조회 정확성 확보

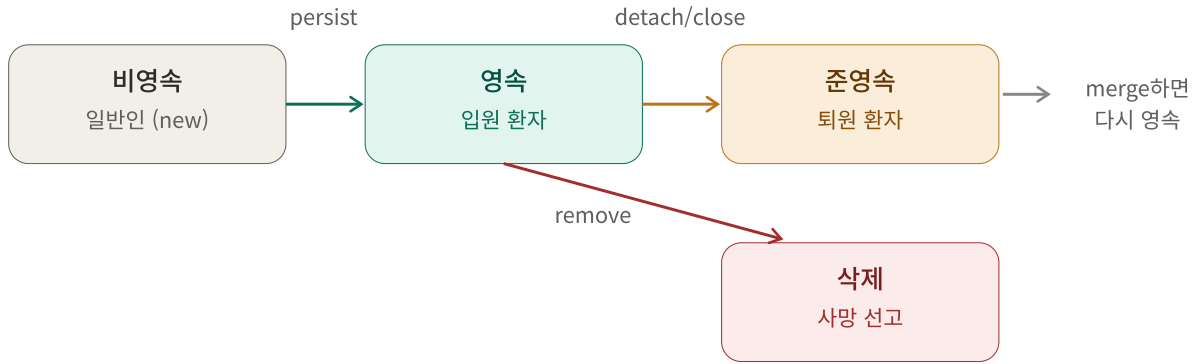
★ 면접 단골: "플러시하면 1차 캐시도 비워지나요?"

정답: 아니요, 절대 안 지워진다. SQL을 DB로 전송(O), 변경 감지로 UPDATE 생성(O)은 하지만 1차 캐시(Map)는 유지. 이유: ①방금 저장하고 바로 조회하는 경우가 많아 캐시 날리면 비효율 ②한 트랜잭션 안 `a == b` 동일성 유지.

메서드	역할	1차 캐시	DB 전송
<code>em.flush()</code>	동기화	유지	O
<code>em.clear()</code>	초기화(비우기)	삭제	X

대량 처리 시 메모리 부족 방지 → 100개 저장 → flush → clear 패턴에 clear 사용.

④ 준영속(Detached) 상태 & merge의 함정 — "비서가 퇴근하면"



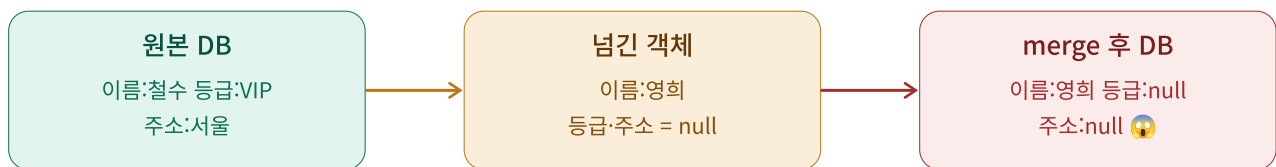
▲ 엔티티 생명주기 4단계

상태	비유	설명
비영속(Transient)	일반인	<code>new</code> 로 만들기만 함. JPA가 모름
영속(Managed)	입원 환자	<code>persist()</code> / <code>find()</code> 로 관리 중. 변경 감지 대상
준영속(Detached)	퇴원 환자	한때 영속이었으나 분리됨. ID는 있지만 관리 안 함
삭제(Removed)	사망 선고	<code>remove()</code> 로 삭제 예약. 커밋 때 DELETE

⚠ **준영속의 위험** 준영속 상태에서 `setName()` 아무리 해도 **변경 감지 안 됨** → DB 반영 X. (비서가 퇴근해 스냅샷 비교 불가) → 1주차 "변경 감지는 트랜잭션 안에서만"과 직결. `em.contains(entity) : true=영속, false=비영속/준영속`.

🔧 merge()의 대참사 — null 덮어쓰기

`merge()` 는 "합치기"가 아니라 "**통째로 덮어쓰기**". ①넘긴 객체의 ID로 DB 원본 조회 → ②넘긴 객체의 **모든 필드**를 원본에 베껴 씀 → ③새 영속 객체 반환(넘긴 객체는 준영속 그대로 버려짐).



merge는 "모든 필드"를 통째로 덮어쓴다 → 비운 칸은 null로 밀어버림

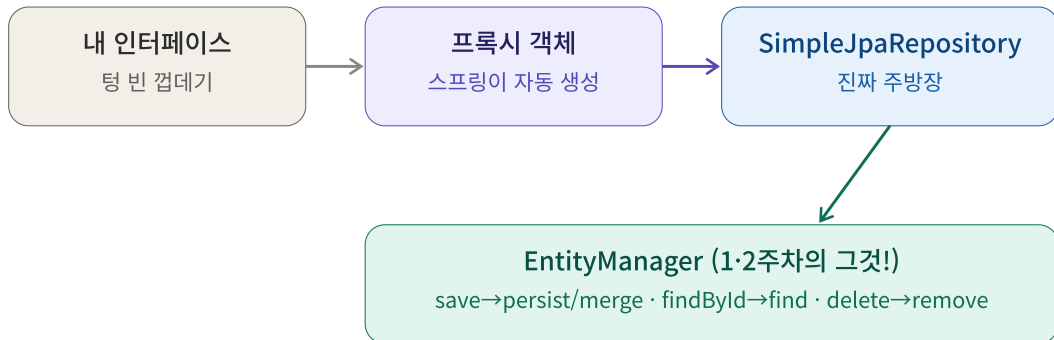
해결: 수정은 merge 말고 변경 감지로. 트랜잭션 안에서 find → 필요한 값만 setName → 트랜잭션 끝날 때 알아서 UPDATE. merge는 통째로 갈아끼우는 특수 상황에만.

▲ 이름만 채워 merge하면 등급·주소가 null로 증발 — 수정은 변경 감지가 정답

⑤ Spring Data JPA — "인터페이스만 만들면 구현은 스프링이 알아서"

```
public interface MemberRepository extends JpaRepository<Member, Long> {
    // 본문 텅 빈. 구현한 적 없음!
}
```

`save()`, `findById()` 를 구현한 적 없는데 동작한다. 비밀: 스프링이 실행 시점에 **구현체(프록시, 가짜 객체)**를 자동 생성해 주입한다.



▲ JpaRepository는 결국 EntityManager 껍데기 — 1주차 원리가 전부 재사용된다

```
public <S extends T> S save(S entity) {
    if (entityInformation.isNew(entity)) {
        em.persist(entity); // 새 거면 persist (INSERT)
    } else {
        return em.merge(entity); // 기존 거면 merge
    }
}
public Optional<T> findById(ID id) {
    return Optional.ofNullable(em.find(domainClass, id)); // em.find()
}
```

그래서 `findById()` 두 번 호출해도 SELECT 한 번은 **1차 캐시** 덕분, 조회 객체 수정만 해도 UPDATE는 **변경 감지** 덕분. (EntityManager를 먼저 배운 이유)

★ save() 함정 두 가지

`save()` 는 `isNew()` 로 판단: **@Id** 가 null이면 **persist**, 값 있으면 **merge**.

함정 1: ID 직접 할당 시 불필요한 SELECT. `@GeneratedValue` 없이 주문번호·UUID를 ID로 넣으면 저장 시 이미 ID가 있음 → `isNew()` 가 "기존 객체" 오판 → **merge** → DB에 SELECT부터 → 성능 저하. 해결: **Persistable** 구현해 `isNew()` 직접 정의(보통 `@CreateDate` 가 null인지로).

```
@Override
public boolean isNew() {
    return createDate == null; // 생성일자 없으면 새 객체 → persist 즉시
}
```

함정 2: save() 반환값 무시하면 수정 누락 버그. **merge** 는 넘긴 객체가 아니라 **새 영속 객체**를 반환한다.

```
memberRepository.save(detachedMember); // (X) 반환값 버림
detachedMember.setName("최종이름"); // 준영속 → DB 반영 안 됨!
```

```
Member managed = memberRepository.save(detachedMember); // (0) 반환값 받기
managed.setName("최종이름"); // 영속 상태 → 변경 감지 동작
```

2주차 한 문장 요약

1주차의 영속성 컨텍스트(엔진) 위에서, **매핑을 안전하게**(EnumType.STRING) 하고, **PK 전략을 이해**(IDENTITY는 쓰기 지연 불가, SEQUENCE는 allocationSize 최적화)하고, **쿼리 나가는 순간(플러시)**을 통제하고, **준영속·merge 함정을 피하고**(수정은 변경 감지로), **JpaRepository가 결국 EntityManager 껍데기임을 꿰뚫어** 보는 것이 목표.

셀프 체크 (복습용)

Q1. IDENTITY 전략은 왜 쓰기 지연이 안 되나?

A. 1차 캐시가 **Map<ID, 엔티티>** 라 ID 필요. AUTO_INCREMENT는 INSERT를 날려야 ID가 나오므로 persist 순간 즉시 INSERT.

Q2. **@Enumerated** 기본값을 그대로 쓰면 왜 위험한가?

A. 기본값 ORDINAL은 순서 숫자로 저장. enum 중간/앞에 값 추가 시 기존 데이터 의미가 밀려 오염. → STRING 사용.

Q3. 플러시하면 1차 캐시가 비워지나?

A. 아니오. SQL만 DB로 전송, 캐시는 유지. 비우는 건 clear().

Q4. 수정할 때 merge를 쓰면 안 되는 이유는?

A. 모든 필드를 통째로 덮어써서, 안 채운 필드가 null로 원본을 지움. → 변경 감지로 수정.

Q5. save() 반환값을 왜 받아야 하나?

A. merge는 새 영속 객체를 반환. 원본(준영속)을 계속 쓰면 변경 감지가 안 됨.